AWS SAA-C03 Study Note: Amazon S3 Advanced Features

This study note covers critical S3 topics for the AWS Certified Solutions Architect - Associate (SAA-C03) exam, focusing on security, access management, and advanced features.

1. S3 Encryption

S3 offers multiple methods to encrypt objects at rest. You must understand the differences, especially regarding key management and audit permissions.

Server-Side Encryption (SSE)

- **SSE-S3 (S3 Managed Keys):**
  - **Keys:** Managed by Amazon S3.
  - **Algorithm:** AES-256.
  - **Rotation:** Handled automatically by S3.
  - **Cost:** No additional cost for keys.
  - **Exam Tip:** The default option if no specific encryption is requested. Simplest to implement.
- **SSE-KMS (KMS Managed Keys):**
  - **Keys:** Managed by AWS Key Management Service (KMS).
  - **Features:** Provides control over key rotation and **audit trails** via CloudTrail (who used the key and when).
  - **Limits:** Accessing objects impacts KMS API quotas (e.g., `GenerateDataKey`, `Decrypt`). Large-scale access can trigger KMS throttling errors.
  - **Exam Tip:** Choose this if the scenario requires **auditability** of key usage or **customer-managed** key rotation policies.
- **SSE-C (Customer-Provided Keys):**
  - **Keys:** You (the customer) manage and store the keys. S3 **does not** store the encryption key.
  - **Process:** You send the key with every request (upload/download). S3 performs the encryption/decryption and then discards the key.
  - **Requirement:** HTTPS is mandatory.
  - **Exam Tip:** Use this if the company security policy dictates that they must manage and hold their own keys exclusively, but they still want AWS to perform the cryptographic operations.
- **DSSE-KMS (Dual-Layer Server-Side Encryption):**
  - Applies two layers of encryption at the server side using KMS keys.
  - Used for strict compliance requirements (e.g., NSA CNSSP 15).

Client-Side Encryption

- You encrypt the data **before** uploading it to S3. S3 never sees the unencrypted data or the keys.

2. S3 Default Encryption

- **Behavior:** S3 buckets now have default encryption enabled automatically (SSE-S3).
- **Configuration:** You can override this default to use SSE-KMS.
- **Effect:** New objects are automatically encrypted. Existing objects remain as they were until they are overwritten or copied.

Default Encryption vs. Bucket Policy

It is crucial to understand the difference between *setting a default* and *enforcing* encryption via policy.

- **Default Encryption (The Safety Net):**
  - **How it works:** If a user uploads an object **without** specifying any encryption headers, S3 automatically applies the bucket's default encryption settings. The upload **succeeds**.
  - **Use Case:** Ensuring no object is ever stored in plain text, without breaking applications that forget to request encryption.
- **Bucket Policy (The Gatekeeper):**
  - **How it works:** You write a policy that explicitly **Denies** `PutObject` requests if they do not include a specific encryption header (e.g., `x-amz-server-side-encryption`).
  - **Result:** If a user tries to upload an unencrypted object, the request **fails** with `403 Access Denied`.
  - **Use Case:** Strictly enforcing compliance where the *client* must be aware of and explicitly request specific encryption (e.g., forcing the use of a specific SSE-KMS key).

3. S3 CORS (Cross-Origin Resource Sharing)

- **Problem:** A web browser blocks requests to an S3 bucket domain (e.g., `mybucket.s3.amazonaws.com`) if the request originates from a different domain (e.g., `www.example.com`).
- **Solution:** Enable CORS on the S3 bucket to allow specific origins to access resources.
- **Configuration:**
  - **AllowedOrigins:** E.g., `https://www.example.com`
  - **AllowedMethods:** E.g., `GET`, `PUT`, `POST`
  - **AllowedHeaders:** E.g., `Authorization`

- **Exam Tip:** If a static website hosted on S3 (or a user on a different domain) receives `403 Forbidden` or "Network Error" when trying to access resources in another bucket, **check CORS configuration**.

4. S3 MFA Delete

- **Purpose:** Enforces Multi-Factor Authentication (MFA) to delete object versions or change the versioning state of a bucket.
- **Requirements:**
  1. **Versioning:** Must be enabled on the bucket.
  2. **Root User:** Only the **Root Account** (bucket owner) can enable/disable MFA Delete.
  3. **CLI/API Only:** Cannot be enabled via the AWS Management Console.
- **Protects Against:**
  - Permanently deleting an object version.
  - Suspending versioning on the bucket.
- **Exam Tip:** If a question asks how to prevent accidental deletion by *privileged* administrators, look for **MFA Delete**. Remember it requires the **Root** user to set up.

5. S3 Access Logs vs. CloudTrail

- **S3 Server Access Logs:**
  - **Scope:** Logs detailed records for requests made to a bucket (e.g., requester, bucket name, request time, request action, response status).
  - **Destination:** Another S3 bucket.
  - **Latency:** Delivery can be delayed (hours).
  - **Best Practice:** Do **not** set the target bucket to be the same as the source bucket (creates a logging loop).
- **CloudTrail Data Events:**
  - **Scope:** Logs API activity (e.g., `GetObject`, `PutObject`).
  - **Cost:** Additional cost for Object-level logging.
  - **Performance:** Faster delivery than S3 Access Logs.
- **Exam Tip:** For **auditing** specific access (who accessed this file?), CloudTrail is often the modern answer. For **detailed access analysis** or troubleshooting server responses, Access Logs are used.

6. S3 Pre-signed URLs

- **Use Case:** Grant temporary access to a private object without changing the bucket policy or making the object public.
- **Mechanism:** Generated using AWS CLI or SDK.
- **Permissions:** The URL inherits the permissions of the **user/role who generated it**.
- **Scenarios:**
  - Allowing a premium user to download a video file.

    - Allowing a user to upload a file directly to S3 without AWS credentials (Pre-signed PUT).
- **Exam Tip:** If the creator of the URL loses permission (e.g., user deleted), the URL stops working immediately, even if it hasn't expired.

7. Glacier Vault Lock & S3 Object Lock (WORM)

Both enforce a **WORM (Write Once, Read Many)** model.

S3 Object Lock

- Applies to individual objects in S3. Requires Versioning.
- **Retention Modes:**
    1. **Governance Mode:** Users generally cannot overwrite/delete, but users with special permissions (`s3:BypassGovernanceRetention`) can. Good for testing or lenient compliance.
    2. **Compliance Mode: Strict.** No one, including the **Root User**, can delete or overwrite the object during the retention period.
- **Legal Hold:** Independent of retention period. Prevents deletion indefinitely until explicitly removed.

Glacier Vault Lock

- Applies to an entire Glacier Vault.
- **Process:** You create a Vault Lock Policy -> Initiate lock -> You have 24 hours to validate -> Complete lock.
- **Exam Tip:** Once the Vault Lock policy is locked (after the 24h window), it **cannot** be changed or deleted. Used for strict, long-term archival compliance (e.g., SEC Rule 17a-4).

8. S3 Access Points

- **Problem:** Managing a single, massive bucket policy for a bucket shared by 100 different teams/applications is complex and risky.
- **Solution:** Create multiple **Access Points** for the bucket.
- **Features:**
    - Each Access Point has its own **policy**.
    - Each Access Point has a unique DNS hostname.
    - Can be restricted to a specific **VPC** (VPC Origin).
- **Exam Tip:** Use Access Points to **decompose** large bucket policies into smaller, application-specific policies.

9. S3 Object Lambda

- **Concept:** Add your own code (Lambda function) to S3 `GET` requests to modify data as it is returned to the application.
- **Workflow:** App -> S3 Object Lambda Access Point -> Lambda Function (Process Data) -> S3 Bucket.
- **Use Cases:**
  - **Redaction:** Removing PII (Personally Identifiable Information) from analytics data for specific users.
  - **Format Conversion:** Converting XML to JSON on the fly.
  - **Watermarking:** Adding dynamic watermarks to images upon download.
- **Exam Tip:** The application does not need to change its code; it just points to the Object Lambda Access Point instead of the bucket directly.